

	RÉPUBLIQUE TUNISIENNE MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE ***** DIRECTION GENERALE DES ÉTUDES TECHNOLOGIQUES ***** INSTITUT SUPÉRIEUR DES ÉTUDES TECHNOLOGIQUES DE CHARGUIA ***** Département Technologies et de l'Informatique	
---	---	---

RAPPORT

De

Projet de Fin d'Études

Licence Appliquée en Technologies de l'Informatique
Parcours : Développement de système d'information

Sujet :

Titre de votre projet de fin d'études

Elaboré par

**Maali Salsabil
&
Abidi Ameni**

Encadré par :

M. Ben Jaffel Zied
M. Saad Houssem

Société d'accueil : Centre informatique de ministère de finances

Année Universitaire : 2025/2026

Table des matières

Introduction générale	1
Chapitre 1 :Cadre général de projet	2
I Présentation de l’organisme d’accueil	2
II Contexte et enjeux	2
III Étude de l’existant	3
III.1 Description de l’existant	3
III.2 Critique de l’existant	4
III.3 Solution proposé	4
IV Spécification des besoins	5
IV.1 Identifications des acteurs	5
IV.2 Besoins fonctionnels	5
IV.3 Besoins non fonctionnels	8
V Choix techniques généraux	9
V.1 Environnement matériel	9
V.2 Environnement logiciel	10
V.3 Bibliothèque utilisée	14
Chapitre 2 :État de l’art	19
I Chatbot Intelligent	19
I.1 Définition et Concepts fondamentaux	19
I.2 Chatbot classique et Chatbot intelligent	19
I.3 Comparaison des différentes types de Chatbots et de leur évolution technologique	20
II Approche Auto-hébergée	21
II.1 Définition de l’approche auto-hébergée	21
II.2 Comparaison entre Auto-hébergée et Cloud	22
III Modèles de langage de grande taille (LLM)	22
III.1 Définition des LLM	22
III.2 Entraînement des modèles de langage LLM	23
III.3 Fonctionnement des modèles de langage LLM	23

III.4	Présentation des principaux modèles de langage (LLM)	24
III.5	Utilisation des LLM dans les systèmes conversationnels	25
IV	RAG(Retrieval-Augmented Generation)	26
IV.1	Définition du RAG	26
IV.2	Principe de fonctionnement d'un système RAG	27
IV.2.1	Requête utilisateur	27
IV.2.2	Recherche documentaire (Retrieval)	27
IV.2.3	Génération de la réponse (Generation)	27
IV.3	Composants d'un système RAG	28
IV.3.1	Les documents	28
IV.3.2	Les embeddings	28
IV.3.3	La base vectorielle	28
IV.3.4	Le LLM (Large Language Model)	28
IV.4	Les types de RAG	29
IV.4.1	RAG naïf(simple RAG)	29
IV.4.2	RAG avec re-ranking	29
IV.4.3	RAG multi-requetes (Multi-query RAG)	29
IV.4.4	RAG hybride (Hybride RAG)	29
IV.4.5	RAG conversationnel (Conversationnel RAG)	30
IV.4.6	RAG agentique (Agentic RAG)	30
IV.5	Comparaison :LLM seul VS LLM+RAG	30
V	Natural Language Processing (NLP)	31
V.1	Définition du NLP	31
V.2	Traitement NLP	31
V.2.1	Segmentation en phrases	31
V.2.2	Tokenisation des mots	31
V.2.3	Stemming	32
V.2.4	Lemmatisation	32
V.2.5	Suppression des stop words	32
V.2.6	Analyse de dépendance	33
V.3	Pipeline NLP classique vs pipeline basé sur LLM	33
Chapitre 3 :Méthodologie de travail		35
I	Méthodologie adoptée	35
I.1	Méthodologie traditionnel	35
I.2	Méthodologie agile	36
I.3	Choix de la méthode à adopter	37
II	Organisation de projet avec Scrum	38
II.1	Rôle de Scrum	38

II.2	Product backlog	40
Chapitre 4 :Analyse et conception du système		41
I	Les Maquettes	41
I.1	Les maquettes de l'administrateur	41
I.2	Les maquettes des agents	44
II	Modélisation UML	49
II.1	Diagramme de cas d'utilisation	49
II.2	Diagramme de séquences	49
II.2.1	Diagramme de séquence conception ""	49
II.2.2	Diagramme de séquence conception ""	49
II.2.3	Diagramme de séquence conception ""	49
II.2.4	Diagramme de séquence conception ""	49
II.2.5	Diagramme de séquence conception ""	49
II.3	Diagramme de Classes	49
II.4	Diagramme de Flux de donnée	49
II.5	Diagramme de Activites	49
III	L'architecture de la solution	49
III.1	architecture général du projet	49
III.2	Patron de conception	50
III.2.1	Modelé d'architecture	50
III.2.2	Les avantages du patron MVC	50
III.3	Diagramme de déploiement	50
III.4	Comparaison des différents types d'architecture logicielle	55

Table des figures

1	Logo FastAPI	10
2	Logo React	10
3	Logo Tailwind	10
4	Logo Python	11
5	Logo Javascript	11
6	Logo de Typescript	11
7	Logo de MongoDB	12
8	Logo de Qdrant	12
9	Logo d'Ollama	12
10	Logo de llama3	12
11	Logo de VS Code	13
12	Logo de Trello	13
13	Logo de Notion	13
14	Logo de Wireframe	13
15	Logo de Draw.Io	13
16	Logo de Docker	14
17	Logo de Ubuntu	14
18	Entraînement des modèles de langage (LLM)	23
19	Fonctionnement des modèles de langage LLM	24
20	Logo ChatGPT	24
21	Logo Claude	25
22	Logo Claude	25
23	Shéma de fonctionnement d'un système RAG	27
24	Shéma de Tokenisation dans NLP	31
25	Shéma de Stemming dans NLP	32
26	Shéma de Lemmatisation dans NLP	32
27	Shéma de Suppression des Stop words dans NLP	33
28	logo de product owner	39
29	logo de scrum master	39
30	logo de développeur	39
31	Shéma de explicatif de framework SCRUM	40

32	Maquette de "Listes des documents"	42
33	Maquette de "Importer document"	43
34	Maquette de "Réclamation"	44
35	Maquette de "Chatbot utilisateur"	45
36	Maquette de "Mes réclamations"	45
37	Maquette de "Suppression de réclamation"	46
38	Maquette de "Nouvelle réclamation"	46
39	Maquette de "Recherche documentaire"	47
40	Maquette de "Résultats de recherche"	47
41	Maquette de "Liste des favoris"	48
42	Maquette de "Notifications"	48

Liste des tableaux

1	Environnement matériel	9
2	Framework et bibliothèque	10
3	Les Langages de Programmation	11
4	Les outils de base de données	12
5	Outil d'IA	12
6	Outils de développement	13
7	Outils de modélisations et conception	13
8	Outils de conteneurisation et système d'exploitation utilisés	14
9	Bibliothèque utilise pour le pipline RAG	15
10	Bibliothèque utilise pour le Backend	16
11	Bibliothèque utilise pour le frontend	17
12	Différence entre Chatbot classique et Chatbot intelligent	20
13	Evolution des Chatbots : de ELIZA à ChatGPT	21
14	Comparaison entre l'approche auto-hébergée et l'approche Cloud	22
15	Comparaison des modèles de langage ChatGPT, Claude et LLaMA	26
16	Comparaison entre LLM et LLM+RAG	30
17	Comparaison entre un pipeline NLP classique et un pipeline basé sur LLM	33
18	Comparaison des principales méthodologies de développement	36
19	Comparaison des principales méthodologies agiles	37
20	Membres de l'equipe Scrum	39
21	Comparaison des différents frameworks Frontend	51
22	Comparaison des technologies Backend	52
23	Comparaison entre MongoDB et PostgreSQL	53
24	Comparaison entre Qdrant, FAISS et ChromaDB	54
25	Comparaison entre Ollama, Claude et GPT	54
26	Comparaison entre l'architecture Microservices et l'architecture n-tiers	55

Introduction générale

Chapitre 1 : Cadre général de projet

Ce chapitre présente le cadre général du projet. Il introduit l'organisme d'accueil, le contexte du projet et la problématique, ainsi que l'étude de l'existant afin d'identifier les limites des solutions actuelles. Il présente également la solution proposée, les besoins fonctionnels et non fonctionnels du système, ainsi que les différents choix techniques adoptés pour la réalisation du projet.

I Présentation de l'organisme d'accueil

Le **Centre Informatique du Ministère des Finances(CIMF)** joue un rôle fondamental dans l'administration publique tunisienne en gérant l'ensemble des systèmes d'information utilisés par les ministères et les institutions publiques. Il est responsable de la gestion et de l'optimisation des infrastructures technologiques qui soutiennent les différents services publics. Une de ses missions majeurs est d'assurer la sécurité numérique au sein de l'administration , en mettant en place des solutions de protection des données et en gérant les certificats électroniques utilisés pour l'authentification et la signature des transactions administratives.

Le CIMF supervise également le développement , le déploiement et la maintenance des applications et plateformes utilisées par les ministères , afin de garantir leur bon fonctionnement et leur conformité aux normes de sécurité. En parallèle , il offre une assistance technique aux utilisateurs et fournit des formations sur l'utilisation des outils numériques. En coordonnant divers projets de digitalisation des services publics , le CIMF contribue activement à l'amélioration de l'efficacité , de la transparence et de la modernisation des processus administratifs, soutenant ainsi les efforts de gouvernance numérique du gouvernement tunisien.

II Contexte et enjeux

Dans les administrations financières publiques, les documents juridiques et réglementaires occupent une place essentielle dans le travail quotidien des agents et des décideurs. Les lois, décrets, circulaires et autres textes administratifs constituent une source indispensable pour l'application des procédures et la prise de décision.

Cependant, l'augmentation continue du volume et de la diversité de ces documents rend

leur gestion de plus en plus complexe. Les méthodes traditionnelles de recherche et de consultation atteignent leurs limites, car l'accès à une information pertinente, fiable et à jour devient souvent long et difficile. Cette situation peut entraîner une perte de temps importante, réduire la productivité des utilisateurs, augmenter le risque d'erreurs dans l'interprétation des textes et impacter la qualité des décisions administratives. De plus, l'absence d'une solution intelligente capable de collecter, structurer, indexer et exploiter automatiquement ces documents constitue un frein à la valorisation du capital informationnel existant.

À cela s'ajoutent des contraintes de sécurité et de confidentialité qui limitent le recours aux solutions cloud externes. Face à ces défis, une question centrale se pose :

Comment concevoir une solution intelligente, sécurisée et hébergée localement permettant la collecte, l'indexation et l'extraction de connaissances à partir des documents juridiques et réglementaires, afin d'offrir une assistance conversationnelle fiable et contextualisée dans le domaine des finances publiques ?

III Étude de l'existant

Cette section présente une analyse de l'existant à travers la description du système actuel et l'identification de ses principales limites. Elle introduit également la solution proposée afin de répondre aux besoins identifiés et d'améliorer l'exploitation des documents juridiques et réglementaires.

III.1 Description de l'existant

Actuellement, la gestion et l'exploitation des documents juridiques et réglementaires au sein des institutions financières publiques reposent principalement sur des méthodes traditionnelles. Les documents utilisés sont de nature variée et comprenant notamment :

- Les lois et de finances ,
- Les décret et arrêtés d'application,
- Les circulaires et notes explicatives,
- Les décisions et interprétations juridiques,

Ces documents sont généralement stockés sous forme de fichiers PDF, Word ou de documents numérisés. Ils sont répartis entre des serveurs internes, des dossiers partagés, des bases d'archive physique ou des plateformes internet. L'accès à l'information s'effectue essentiellement par une consultation manuelle des documents ou par une recherche simple basée sur des mots-clés et une organisation hiérarchique des dossiers.

Ainsi, malgré la richesse et la diversité des ressources documentaires disponibles, leur exploitation demeure complexe, peu structurée et fortement dépendante de l'intervention humaine.

III.2 Critique de l'existant

Bien que les méthodes actuelles présentent l'avantage d'un stockage centralisé minimale et d'un accès contrôlé aux documents, elles souffrent de plusieurs limites importantes qui réduisent leur efficacité.

Ces limites se manifestent notamment par :

- Le volume très élevé des documents juridiques et réglementaire , rendant leur consultation manuelle lente et peu efficace ,
- L'absence d'une solution automatique de collecte , de structuration et d'indexation intelligente des documents,
- Une recherche essentiellement basée sur mots-clés , sans prise en compte du contexte juridique ni des relations entre les textes ,
- Une difficulté à retrouver rapidement des réponses précises à des questions complexes ou transversales ,
- Une dépendance excessive à l'expertise individuelle, ce qui peut entraîner des erreurs d'interprétation ou des incohérences,
- L'absence d'outils d'extraction automatique de connaissance à partir des documents existants

De plus, les solutions cloud basés sur des services d'IA externes ne sont généralement pas adoptés au contexte des administrations publiques, en raison des contraintes liées à la confidentialité des données, à la souveraineté nationale et à la sécurité des informations sensibles.

III.3 Solution proposé

Pour répondre à cette problématique, le projet propose la mise en place d'un chatbot intelligent auto-hébergé basé sur un pipeline **RAG(Retrieval Augmented Generation)**. Cette approche permet de combiner la puissance des modèles de langage avec une base documentaire contrôlée et fiable.

La solution proposée repose sur les axes suivants :

- **Collecte et structuration des documents** : automatisation de l'ingestion des documents juridiques et réglementaires, conversion en formats exploitables et classification par type et par thématique ,
- **Indexation vectorielle** : transformation des documents en embeddings sémantiques et stockage dans une base vectorielle afin de permettre une recherche contextuelle efficace,
- **Pipeline RAG** : récupération des passages pertinents à partir de la base documentaire et génération de réponses par un modèle de langage (**LLM**) open source hébergé localement,

- **Chatbot conversationnel** : interface web permettant aux utilisateurs de poser des questions en langage naturel et d'obtenir des réponses basées exclusivement sur les documents internes,
- **Sécurité et souveraineté** : déploiement local sécurisé sans dépendance à des services cloud externes,
- **Evaluation des performances** : mesure de la précision, de la pertinence et du temps de réponse du système.

Cette solution vise à transformer une base documentaire statique en une véritable source de connaissance exploitable en temps réel.

IV Spécification des besoins

IV.1 Identifications des acteurs

La solution proposée implique plusieurs acteurs qui interagissent avec le système dans le cadre de son utilisation et de sa gestion. Ces acteurs représentent les différents profils d'utilisateurs qui exploitent l'application selon leurs besoins et leurs responsabilités au sein des administrations financières publiques.

- **Administrateur** : Il est responsable de la gestion globale du système. Il assure l'importation et la gestion des documents juridiques, supervise l'indexation, contrôle les activités des utilisateurs, gère les réclamations et garantit la sécurité ainsi que la traçabilité des opérations,
- **Utilisateur (agent financier)** : Il utilise le système pour effectuer des recherches, poser des questions en langage naturel via le chatbot et exploiter les informations dans le cadre de ses tâches quotidiennes. Il peut également gérer ses conversations et soumettre des réclamations si nécessaire.

IV.2 Besoins fonctionnels

- **Gestion des documents** :
 - **Importation des documents** :
 - L'administrateur peut importer des documents juridiques sous formats PDF ou Word ,
 - Le système peut nettoyer automatiquement les documents importés,
 - Le système peut découper les documents en segments pertinents et les vectoriser,
 - Le système peut préremplir automatiquement le titre du document,
 - L'administrateur peut sélectionner une catégorie lors de l'importation,

- L’administrateur peut visualiser les documents avant validation,
- L’administrateur peut suivre la progression de l’indexation,
- L’administrateur peut annuler l’indexation d’un document.
- **Consultation des documents :**
 - L’administrateur peut consulter la liste des documents indexés,
 - L’administrateur peut consulter les détails d’un document,
 - L’administrateur peut filtrer les documents par catégorie ou status
 - L’administrateur peut réinitialiser les filtres,
 - L’administrateur peut rechercher un document par son titre,
 - L’administrateur peut exporter la liste des documents en PDF ou Excel,
 - L’utilisateur peut filtrer les documents par catégorie ou date
- **Gestion des documents :**
 - L’administrateur peut modifier les informations d’un document indexé ,
 - L’administrateur peut supprimer un document de la base vectorielle.
- **Recherche et consultation intelligente :**
 - **Interaction intelligente (Chatbot) :**
 - L’utilisateur peut poser une question en langage naturel,
 - Le système analyse la question et retourne une réponse basée sur les documents indexés,
 - La réponse est générés à partir des contenus les plus pertinents de la base documentaire.
 - **Recherche par mots-clés :**
 - L’utilisateur peut effectuer une recherche en saisissant un ou plusieurs mots-clés,
 - Le système affiche les documents contenant les termes recherchés avec mise en évidence,
 - L’utilisateur peut consulter ses dernières recherches.
 - **Résultats de recherche :**
 - L’utilisateur peut visualiser la liste des documents trouvés,
 - Le système met en évidence les passages contenant les termes recherchés,
 - L’utilisateur peut consulter un aperçu du document,
 - L’utilisateur peut consulter le document complet.
 - **Gestion des documents issus de la recherche :**
 - L’utilisateur peut ajouter un document à ses favoris,
 - L’utilisateur peut consulter sa liste des favoris,

- L'utilisateur peut retirer un document de ses favoris.
- **Gestion des conversations :**
 - **Consultation et organisation :**
 - L'utilisateur peut consulter, ses anciennes conversations,
 - L'utilisateur peut rechercher dans une conversation,
 - L'utilisateur peut renommer une conversation,
 - L'utilisateur peut supprimer une conversation.
 - **Archivage :**
 - L'utilisateur peut archiver et restaurer une conversation,
 - L'utilisateur peut consulter ses conversations archivées
 - **Actions sur les réponses :**
 - L'utilisateur peut copier une réponse,
 - L'utilisateur peut télécharger une réponse en PDF,
 - L'utilisateur peut exporter une conversation en PDF,
 - L'utilisateur peut choisir un mode de réponse court ou détaillé.
- **Gestion des comptes et sécurité :**
 - **Authentification :**
 - L'utilisateur peut se connecter,
 - L'utilisateur peut se déconnecter,
 - **Gestion du profil :**
 - L'utilisateur peut se modifier ses informations personnelles
 - **Sécurité :**
 - Le système garantit la sécurité des sessions utilisateurs,
 - **Interface :**
 - L'utilisateur peut activer un mode sombre.
- **Evaluation et amélioration :**
 - L'utilisateur peut évaluer la qualité des réponses,
 - L'administrateur peut consulter des statistiques d'utilisation.
- **Traçabilité et audit :**
 - **Suivi des activités :**
 - L'administrateur peut consulter les activités des utilisateurs,
 - Le L'administrateur peut consulter les détails d'activité d'un utilisateur,
 - Le système peut filtrer les activités par utilisateur ou type d'action.

- **Historique :**
 - Le système peut enregistrer les conversations avec leurs horodatages.
- **Exportation :**
 - L’administrateur peut exporter les journaux d’activité en JSON, Excel ou PDF.
- **Gestion des réclamations :**
 - **Coté utilisateur :**
 - L’utilisateur peut envoyer une réclamation ,
 - L’utilisateur peut consulter la liste de ses réclamations,
 - L’utilisateur peut filtrer ses réclamations par statut,
 - L’utilisateur peut rechercher une réclamation,
 - L’utilisateur peut consulter le détail d’une réclamation,
 - peut voir la progression de sa réclamation.
 - **Coté administrateur :**
 - L’administrateur peut consulter toutes les réclamations,
 - L’administrateur peut filtrer les réclamations par statut, catégorie ou urgence ,
 - L’administrateur peut rechercher une réclamation ,
 - L’administrateur peut consulter une réclamation,
 - L’administrateur peut répondre à une réclamation.
- **Notifications :**
 - L’utilisateur peut recevoir une notification lorsqu’un document est indexé,
 - L’utilisateur peut recevoir une notification lors de la réponse d’une réclamation.

IV.3 Besoins non fonctionnels

- **Traçabilité :**
 - Les réponses doivent être accompagnées des références documentaires utilisés,
 - Les actions des utilisateurs doivent être enregistrer pour l’audit.
- **Ergonomie et utilisabilité :**
 - Interface doit être intuitive et simple à l’utilisation,
 - Les réponses doivent être claires, structurées et compréhensibles.
- **Maintenabilité :**
 - Le système doit être permettre l’ajout et la suppression de document sans interruption majeur,

- L'architecture doit être modulaire pour faciliter la mise à jour.
- **Fiabilité :**
 - Le système doit garantir la disponibilité continue du service,
 - Les réponses générées doivent être basées exclusivement sur les documents internes (locale).
- **Confidentialité :**
 - Aucun donné ne doit être transmise vers des services cloud externes ,
 - Le modèle et la base documentaire doivent être déployés localement,
 - Les données sensibles doivent être protégées contre tout accès non autorisés.
- **Performance :**
 - La recherche documentaire et la génération de réponses doivent être optimisés pour éviter toute latence excessive,
 - L'indexation des documents doit être réalisé de manière efficace afin de garantir la rapidité pour la recherche

V Choix techniques généraux

Dans cette section , nous présentons l'environnement de développement ainsi que les choix techniques adoptés pour la réalisation du projet. Elle décrit les ressources matérielles utilisées ainsi que les outils et technologies logicielles. Cette partie permet de justifier les choix effectués en fonction des besoins du projet.

V.1 Environnement matériel

Dans cette partie, nous présentons les spécifications matérielles des équipements utilisés dans le cadre du développement du projet.

TABLE 1 – Environnement matériel

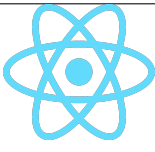
Composant	Hp Laptop	MSI Thin GF63
Processeur	Intel Core i5-1334U (13 génération, 1,30 GHz)	Intel Core i5-12450H (12 génération, 2.00 GHz)
RAM	32 Go	32 Go
Stockage	512 Go	512 Go
Carte Graphique	2 Go	4 Go
Système d'exploitation	Windows 11 Famille (Version 10.0.26200)	Windows 11 Pro (Version 25H2)

V.2 Environnement logiciel

Dans le cadre de notre projet, nous avons sélectionné un ensemble de technologies et d'outils en nous basant sur une étude comparative présentée en (annexe 1). Ce choix vise à garantir la performance, la flexibilité et la scalabilité de l'application. Les technologies adoptées ont été regroupées selon leur rôle dans l'architecture du système, notamment les langages de programmation, les frameworks, les bases de données, ainsi que les outils de développement et de déploiement.

- **Framework et bibliothèque utilisés**

TABLE 2 – Framework et bibliothèque

Logo	Technologie	Description
 Figure 1 – Logo FastAPI	FastApi	Framework web performant et moderne en Python, qui facilite la création d'API grace aux annotations de type standard.[1]
 Figure 2 – Logo React	React	Bibliothèque Javascript utilisée pour construire des interfaces utilisateur dynamiques et réactives. Elle repose sur des composants réutilisables et améliore l'expérience utilisateur.[2]
 Figure 3 – Logo Tailwind	Tailwind CSS	Framework CSS utilitaire permettant de concevoir rapidement des interfaces modernes et responsives. il offre des classes prêtes à l'emploi pour styliser les composants sans écrire beaucoup de CSS.[3]

- **Langage de Programmation**

TABLE 3 – Les Langages de Programmation

Logo	Technologie	Description
 Figure 4 – Logo Python	Python	Langage de Programmation polyvalent, largement utilisé dans le développement backend et les applications d'intelligence artificielle. Il offre une syntaxe simple et de nombreuses bibliothèques pour le traitement de données et le machine learning.[4]
 Figure 5 – Logo Javascript	Javascript	Langage de programmation utilisé principalement pour le développement web coté client . Il permet de créer des interfaces interactives et dynamiques dans les applications web.[5]
 Figure 6 – Logo de Typescript	Typescript	Langage basé sur JavaScript qui ajoute le typage statique. Il permet d'améliorer la qualité du code, la détection des erreurs et la maintenabilité des applications complexes.[6]

- Base de données et stockage

TABLE 4 – Les outils de base de données

Logo	Technologie	Description
 Figure 7 – Logo de MongoDB	MongoDB	Base de données NoSQL orientée documents, qui stocke les données sous forme JSON. Elle offre une grande flexibilité pour gérer des données non structurées.[7]
 Figure 8 – Logo de Qdrant	Qdrant	Base de données vectorielle utilisée pour stocker et rechercher des embeddings. Elle permet d'effectuer des recherches sémantiques efficaces dans les systèmes basés sur l'IA.[8]

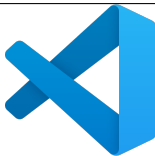
- Intelligence Artificielle

TABLE 5 – Outil d'IA

Logo	Technologie	Description
 Figure 9 – Logo d'Ollama	Ollama	Plateforme permettant d'exécuter des modèles de langage (LLM) en local. Elle garantit la confidentialité des données et facilite l'intégration de l'IA dans les applications.[9]
 Figure 10 – Logo de llama3	Llama3	[10]

- Outils de développement

TABLE 6 – Outils de développement

Logo	Technologie	Description
 Figure 11 – Logo de VS Code	Vs Code	Éditeur de code léger et puissant, utilisé pour le développement d'applications. Il propose de nombreuses extensions pour améliorer la productivité.[11]
 Figure 12 – Logo de Trello	Trello	Outil de gestion de projet basé sur un système de tableaux et de cartes. Il permet de suivre l'avancement des tâches, organiser le travail en équipe et gérer les sprints de manière visuelle.[12]
 Figure 13 – Logo de Notion	Notion	Outil de gestion et d'organisation permettant de centraliser les notes et la documentation. Il facilite la collaboration et la structuration des informations du projet .[13]

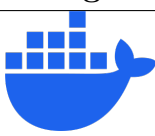
- **Modélisation et Conception**

TABLE 7 – Outils de modélisations et conception

Logo	Technologie	Description
 Figure 14 – Logo de Wireframe	Wireframe	Un wireframe est un schéma en noir et blanc représentant la structure et l'architecture fonctionnelle d'une page web ou d'une application.[14]
 Figure 15 – Logo de Draw.io	Draw.io	Outil permettant de créer des diagrammes UML, des schémas d'architecture et des représentations visuelles du système.[15]

- **Conteneurisation et système**

TABLE 8 – Outils de conteneurisation et système d’exploitation utilisés

Logo	Technologie	Description
 Figure 16 – Logo de Docker	Docker	Permet de créer des conteneurs isolés et reproductibles pour déployer et gérer facilement l’application .[16]
 Figure 17 – Logo de Ubuntu	Ubuntu	Fournit un environnement serveur stable, sécurisé et compatible pour héberger les applications Dockerisées.[17]

V.3 Bibliothèque utilisée

- **Bibliothèque utilise pour le Pipeline RAG** Voici les principales bibliothèques utilisées dans le pipeline RAG pour le traitement des documents ,la génération des embeddings ,la recherche sémantiques et l’interaction avec le modèles de la langage .

TABLE 9 – Bibliothèque utilise pour le pipline RAG

Bibliothèque	Description
qdrant-client	Client Python utilisé pour communiquer avec Qdrant afin de stocker et rechercher les embeddings vectoriels.
sentence-transformers	Génération d'embeddings textuels à partir de modèles Transformer pour la recherche sémantique.
transformers	Utilisation des modèles NLP et LLM disponibles sur Hugging Face.
torch	Framework de calcul utilisé pour l'exécution des modèles d'intelligence artificielle.
requests	Envoi de requêtes HTTP vers Ollama pour la génération de réponses et d'embeddings.
pypdf	Extraction du texte depuis les fichiers PDF.
python-docx	Lecture et extraction du contenu des fichiers Word .docx.
scikit-learn	Outils de traitement de données et de calcul de similarité pour le NLP.
numpy	Manipulation de matrices, vecteurs et calcul numérique.
scipy	Bibliothèque scientifique utilisée avec les traitements mathématiques et statistiques.
tokenizers	Tokenisation optimisée des textes pour les modèles NLP.
huggingface_hub	Téléchargement et gestion des modèles IA depuis Hugging Face.
safetensors	Chargement sécurisé des poids des modèles d'intelligence artificielle.
tqdm	Affichage des barres de progression lors des traitements et de l'indexation.

- **Bibliothèque utilise pour le Backend** Le tableau suivant présente les bibliothèques utilisées dans le développement du backend générale, notamment pour la création des API, la gestion des données et la configuration du système.

TABLE 10 – Bibliothèque utilise pour le Backend

Bibliothèque	Description
fastapi	Framework principal utilisé pour développer les API REST du système.
uvicorn	Serveur ASGI utilisé pour exécuter l'application FastAPI.
starlette	Framework ASGI interne servant de base à FastAPI.
pydantic	Validation des données et création des schémas de données.
pydantic-settings	Gestion des configurations et variables d'environnement.
pymongo	Connexion et manipulation de la base de données MongoDB.
python-dotenv	Chargement des variables d'environnement depuis un fichier .env.
python-multipart	Gestion des fichiers envoyés via formulaires et uploads.
httpx	Client HTTP asynchrone utilisé pour les communications externes.
Jinja2	Génération de templates HTML côté serveur.
lxml	Parsing et traitement de documents XML et HTML.
PyYAML	Lecture et écriture des fichiers YAML.
rich	Amélioration de l'affichage et du logging dans la console.
typer	Création d'interfaces de ligne de commande (CLI).
black	Outil automatique de formatage du code Python.
ruff	Lint pour l'analyse et la qualité du code Python.
mypy	Vérification statique des types dans le code Python.

- **Bibliothèque utilise pour le Frontend** Le tableau ci-dessous présente regroupe les principales bibliothèques utilisées dans le développement du frontend afin de concevoir l'interface utilisateur, gérer la navigation et intégrer les différentes fonctionnalités interactives.

TABLE 11 – Bibliothèque utilise pour le frontend

Bibliothèque	Description
react	Bibliothèque principale utilisée pour construire l'interface utilisateur.
react-dom	Gestion du rendu des composants React dans le navigateur.
react-router-dom	Gestion des routes et de la navigation entre les pages.
vite	Outil de build rapide et serveur de développement frontend.
typescript	Ajout du typage statique au JavaScript.
tailwindcss	Framework CSS utilitaire utilisé pour le design de l'interface.
@tailwindcss/vite	Intégration de Tailwind CSS avec Vite.
lucide-react	Bibliothèque d'icônes modernes pour l'interface utilisateur.
apexcharts	Création de graphiques et visualisations de données.
react-apexcharts	Intégration d'ApexCharts avec React.
jspdf	Génération de fichiers PDF côté client.
jspdf-autotable	Création automatique de tableaux dans les fichiers PDF.
xlsx	Importation et exportation de fichiers Excel.
pdfjs-dist	Lecture et prévisualisation des fichiers PDF dans le navigateur.
eslint	Analyse et contrôle de la qualité du code JavaScript/TypeScript.
@vitejs/plugin-react	Plugin React utilisé avec Vite.
@types/react, @types/react-dom, @types/node	Définitions de types TypeScript pour React, React DOM et Node.js.

Dans ce premier chapitre, nous avons présenté le cadre générale de projet en introduisant l'organisme d'accueil ainsi que son rôle dans la gestion des systèmes d'information au sein des administrations financières publiques. Nous avons ensuite exposé le contexte et les enjeux liés à la gestion des documents juridiques et réglementaires, en mettant en évidence les difficultés rencontrées dans leur exploitation. L'étude de l'existant a permis d'analyser les méthodes actuellement utilisées ainsi que leurs limites, ce qui a conduit à la formulation de la problématique du projet. Afin de répondre à cette problématique, une solution basée sur un Chatbot intelligent utilisant une approche RAG a été proposée. Par la suite, nous avons défini le périmètre fonctionnel du système à travers l'identification des besoins fonctionnels et non fonctionnels. Enfin, nous avons présenté les choix techniques

adoptés ainsi que l'environnement matériel et logiciel utilisé pour la réalisation du projet. Ce chapitre constitue ainsi une base essentielle pour la suite du travail, qui sera consacrée à la conception détaillée et à l'implémentation de la solution proposée.

Chapitre 2 : État de l'art

Ce chapitre présente les principaux concepts liés aux chatbots intelligents et à l'intelligence artificielle conversationnelle. Il couvre les modèles de langage de grande taille (**LLM**) ainsi que les approches de déploiement auto-hébergées. Une attention particulière est portée à la technique RAG, qui combine la recherche d'information et la génération de texte afin d'améliorer la fiabilité des réponses. Ce chapitre fournit les bases théoriques nécessaires à la compréhension du projet.

I Chatbot Intelligent

I.1 Définition et Concepts fondamentaux

Un Chatbot intelligent est un agent conversationnel fondé sur des techniques d'intelligence artificielle, qui comprend et traite le langage naturel des utilisateurs pour formuler des réponses adaptées. Il s'appuie principalement sur des technologies telles que le traitement du langage naturel (**NLP**) pour analyser les requêtes, la génération du langage naturel (**NLP**) pour produire des réponses cohérentes, ainsi que l'apprentissage automatique pour améliorer ses performances au fil des interactions. Ces mécanismes permettent au Chatbot intelligent de comprendre le sens et le contexte des conversations et d'évoluer progressivement grâce à l'expérience acquise.[18]

I.2 Chatbot classique et Chatbot intelligent

La différence entre un Chatbot classique et un Chatbot intelligent réside avant tout dans leur aptitude à comprendre et s'adapter. Le Chatbot classique fonctionne sur la base de règles et de scénarios prédéfinis, en utilisant le plus souvent des techniques de correspondance de mots-clés. Cette approche restreint sa capacité à gérer des requêtes complexes ou imprévues. A l'inverse, le Chatbot intelligent s'appuie sur des techniques avancées d'intelligence artificielle, notamment le traitement du langage naturel et l'apprentissage automatique, ce qui lui permet de comprendre le sens des requêtes, de tenir compte du contexte conversationnel et d'adapter dynamiquement des réponses. Il peut, en outre, perfectionner ses performances au cours du temps, en s'apprenant à partir des échanges avec les utilisateurs.[19]

TABLE 12 – Différence entre Chatbot classique et Chatbot intelligent

Critère	Chatbot classique	Chatbot intelligent
Fonctionnement	Basé sur des règles fixes et scripts	Basé sur l'intelligence artificielle
Compréhension	Correspondance de mots-clés	Compréhension du langage naturel
Contexte	Non pris en compte	Pris en compte
Adaptabilité	Faible	Élevée
Apprentissage	Aucun	Apprentissage automatique
Complexité	Requêtes simples	Requêtes complexes

I.3 Comparaison des différents types de Chatbots et de leur évolution technologique

La comparaison des différents types de Chatbots et de leur évolution technologique repose sur plusieurs critères liés à leur fonctionnement, leur capacité de compréhension et leur niveau d'intelligence. Le tableau suivant présente une synthèse des principales caractéristiques des Chatbots, depuis les premiers systèmes comme ELIZA jusqu'aux modèles avancés tels que ChatGPT en se basant sur des critères.

TABLE 13 – Evolution des Chatbots : de ELIZA à ChatGPT

Critère	ELIZA	SmarterChild	Siri	ChatGPT
Année d'apparition	1966	2001	2011	2022
Approche	Basée sur des règles	Règles + base de données	IA + NLP+ reconnaissance vocale	IA générative (LLM)
Compréhension du langage	Très limitée	Limitée	Avancée	Très avancée
Type de réponse	Reformulation de phrases	Réponses prédéfinies	Réponses contextuelles	Réponses générées dynamiquement
Apprentissage	Aucun	Aucun	Limité	Apprentissage à grande échelle
Interaction	Textuelle	Textuelle	Vocale et textuelle	Multimodale
Capacité décisionnelle	Non	Très limitée	Oui (actions simples)	Oui (raisonnement et assistance)
Domaine de connaissance	Très restreint	Restreint	Général + services système	Très large et multi-domaines
Objectif principal	Simulation de dialogue	Fourniture d'informations	Assistant personnel	Assistant conversationnel intelligent
Type de Chatbot	Classique	Classique amélioré	Intelligent	Intelligent avancé

II Approche Auto-hébergée

II.1 Définition de l'approche auto-hébergée

L'approche auto-hébergée (self-hosting) correspond à un modèle de déploiement par lequel une organisation ou un individu héberge, exécute et administre ses applications et services sur une infrastructure dont il a la mainmise directe. Ce modèle autorise une gestion autonome des ressources et un contrôle accru sur les données, la sécurité et la configuration des systèmes, contrairement aux solutions de Cloud Computing où ces éléments sont gérés par des fournisseurs tiers.

II.2 Comparaison entre Auto-hébergée et Cloud

Le choix entre une approche auto-hébergée et une solution Cloud repose sur plusieurs critères techniques, organisationnels et économiques . Le tableau suivant présente une comparaison synthétique entre les deux modèles selon les principaux aspects pertinents pour un système informatique intelligent.

TABLE 14 – Comparaison entre l’approche auto-hébergée et l’approche Cloud

Critère	Approche auto-hébergée	Approche Cloud
Sécurité	Contrôle total des mécanismes de sécurité et des accès	Sécurité principalement gérée par le fournisseur
Confidentialité	Données conservées en interne	Données hébergées chez un tiers
Souveraineté des données	Maîtrise complète de la location et du traitement des données	Dépendance à la juridiction du fournisseur
Coût	Investissement initial élevé (matériel , maintenance)	Coût variable basé sur l’usage
Maintenance	Assurée par l’organisation	Assurée par le fournisseur Cloud
Scalabilité	Limitée par les ressources internes	Élevée et flexible
Dépendance au fournisseur	Faible dépendance	Forte dépendance au prestataire Cloud

III Modèles de langage de grande taille (LLM)

III.1 Définition des LLM

Un modèle de langage de grande taille est un type de modèle d’intelligence artificielle. Il utilise des techniques d’apprentissage automatique pour traiter et générer du texte. Ce modèle fonctionne en utilisant les probabilités de cooccurrence entre les mots. Cela signifie qu’il regarde comment les mots sont utilisés ensemble dans le langage .

Ce modèle est entraîné sur des quantités énormes de texte. Il apprend à prédire chaque mot d’une séquence en fonction des mots qui viennent avant. Cela lui permet de produire des textes qui ont du sens et qui ressemblent à la façon dont les humains parlent et écrivent. Les textes générés par ce modèle sont souvent très réalistes et difficiles à distinguer de ceux écrits par des humains.[20]

III.2 Entraînement des modèles de langage LLM

L'entraînement des modèles de langage de grande taille est une étape très importante pour leur développement. Cet entraînement a lieu avant de les utiliser et consiste à utiliser de grandes quantités de données textuelles. Ces données proviennent de livres, d'articles scientifiques et de contenus disponibles sur le web.

Le modèle apprend principalement de manière auto-supervisée. Cela signifie qu'il est entraîné pour prédire le mot suivant dans une phrase donnée. Au fur et à mesure de son apprentissage, le modèle comprend les règles de grammaire, les relations entre les mots et les particularités du langage naturel. En analysant un grand nombre de phrases, le modèle devient capable de reconnaître des modèles linguistiques complexes et d'appliquer ses connaissances à de nouvelles situations.

Certaines méthodes incluent également une étape appelée apprentissage par renforcement avec des commentaires humains. Cette étape permet d'améliorer la qualité et la sécurité des réponses générées par le modèle.

L'entraînement est donc une phase cruciale, car il influence directement les performances et les capacités du modèle lorsqu'il est utilisé par la suite.

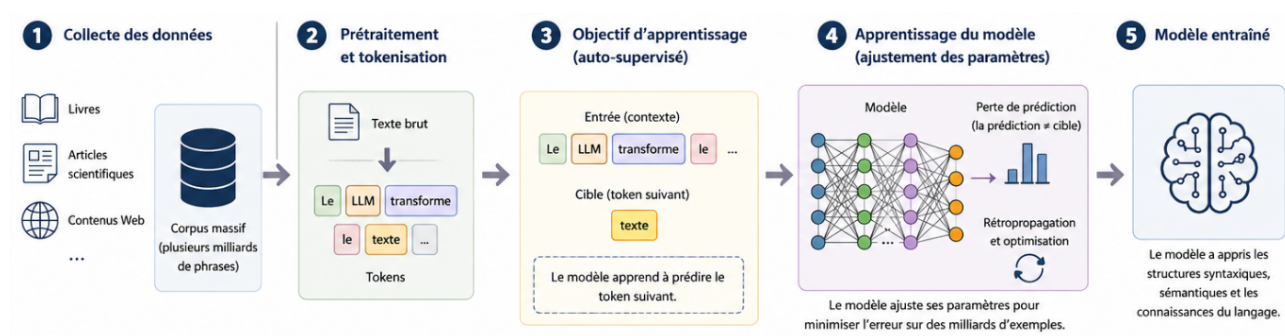


Figure 18 – Entraînement des modèles de langage (LLM)

III.3 Fonctionnement des modèles de langage LLM

Le fonctionnement d'un modèle de langage de grande taille repose sur plusieurs étapes. Ces étapes permettent de transformer une entrée textuelle en une réponse générée automatiquement.

- **Saisie du texte de l'utilisateur** : Le processus commence par la réception d'une entrée textuelle fournie par l'utilisateur. Cette entrée peut être une question ou une instruction.
- **Tokenisation** : Le texte est ensuite divisé en unités élémentaires appelées tokens. Ces tokens peuvent être des mots, des parties de mots ou des caractères. Cette transformation permet de convertir le texte en une représentation que le modèle peut utiliser.

- **Analyse du contexte** : Les tokens sont analysés à l'aide de l'architecture **Transformer** , qui utilise des mécanismes d'attention. Cette étape permet de comprendre les relations entre les tokens et le contexte global de la séquence.
- **Prédiction du token suivant** : Le modèle prédit le token suivant le plus probable en fonction du contexte analysé. Il utilise les probabilités apprises lors de la phase d'entraînement pour faire cette prédiction.
- **Génération de la réponse** : Le processus de prédiction est répété plusieurs fois . Chaque token généré est ajouté à la séquence, permettant au modèle de construire progressivement une réponse. Cette réponse est cohérente et adaptée à la requête initiale de l'utilisateur

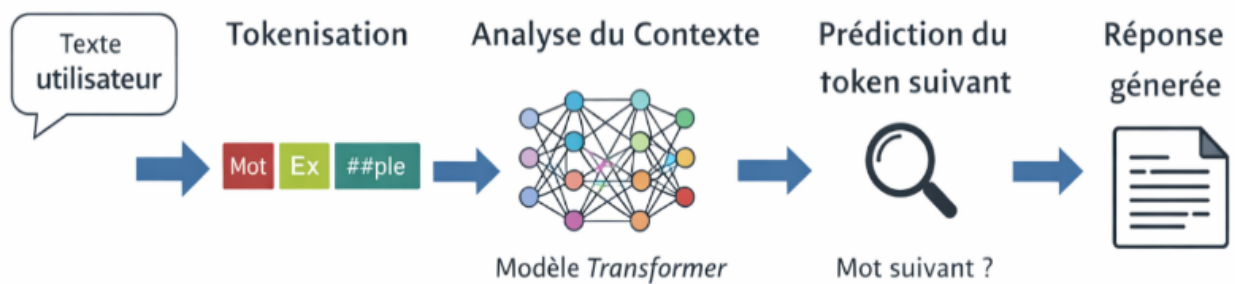


Figure 19 – Fonctionnement des modèles de langage LLM

III.4 Présentation des principaux modèles de langage (LLM)

- **ChatGPT** : est un modèle de langage de grande taille créé par OpenAI. Il utilise l'architecture **Transformer** pour produire des textes qui ont du sens et qui sont adaptés au contexte. Pour fonctionner, il a d'abord été entraîné sur des grandes quantités de données, puis il a été amélioré grâce à des méthodes d'apprentissage automatique qui impliquent des retours humains. Cela lui permet de fournir des réponses de meilleure qualité, plus pertinentes et plus fluides. [21]



Figure 20 – Logo ChatGPT

- **Claude** : est un modèle de langage créé par **Anthropic**. Il est conçu pour fournir des réponses fiables et sûres, tout en respectant des principes éthiques stricts. Claude se concentre sur la réduction des biais d'apprentissage supervisé et par renforcement.

Ces méthodes lui permettent de mieux comprendre ce que l'utilisateur veut dire et d'améliorer la qualité des réponses qu'il fournit.[22]



Figure 21 – Logo Claude

- **LLaMA** : est une famille de modèles de langage développée par Meta. Ces modèles sont conçus pour offrir un bon équilibre entre performance et efficacité en termes de calcul. Contrairement à certains modèles qui appartiennent à des entreprises privées, LLaMA est conçu pour être plus accessible et adaptable. Cela signifie qu'il peut être utilisé dans des situations où les ressources informatiques sont limitées ou lorsqu'il est nécessaire de l'installer localement .[23]



Figure 22 – Logo Claude

III.5 Utilisation des LLM dans les systèmes conversationnels

Les modèles de langage de grande taille comme ChatGPT, Claude et LLaMA ont des architectures similaires. Mais ils sont différents en termes d'objectifs, de performances et de domaines d'application. Plusieurs études académiques ont examiné ces différences. Elles ont mis en évidence des variations en termes de fiabilité, de flexibilité et d'orientation d'usage.

Table 15 – Comparaison des modèles de langage ChatGPT, Claude et LLaMA

Critère	ChatGPT	Claude	LLaMA
Fonctionnalités	Offre une génération de contenu dynamique et cohérent	Se caractérise par une génération fiable et sécurisée	présente une flexibilité et une adaptabilité notables, ainsi que la possibilité d'un déploiement local
Méthodologie d'entraînement	entraîné à partir de données massives et utilise la rétroaction humaines pour améliorer ses performances	entraîné en utilisant une approche de rétroaction humaine orientée sécurité	a bénéficié d'un pré-entraînement optimisé pour améliorer ses capacités
Performance	offre une haute fluidité et sa cohérence	se distingue par sa fiabilité et sa cohérence	présente une bonne performance, mais celle-ci peut varier en fonction de la configuration
Précision des résultats	offre une précision élevée, mais peut parfois commettre des erreurs	très fiable et commet moins d'erreurs	présente une précision variable, dépendant du réglage utilisé
Cas d'utilisation	est bien adapté pour les services client et les assistants virtuels	convient particulièrement aux domaines de la santé, de l'éducation et des domaines sensibles	est approprié pour les environnements à ressources limitées, les applications mobiles et IoT

IV RAG(Retrieval-Augmented Generation)

IV.1 Définition du RAG

La génération augmentée de récupération (Retrieval-Augmented Generation) est une méthode qui consiste à ajouter une étape de recherche d'informations pour extraire des contenus pertinents à partir d'un corpus documentaire. Le RAG utilise ces informations combinées à la requête de l'utilisateur pour permettre au modèle de LLM de générer une réponse plus précise et plus contextualisée.[24]

IV.2 Principe de fonctionnement d'un système RAG

Un système de RAG fonctionne selon une architecture à plusieurs étapes, qui permet de transformer une requête utilisateur en une réponse contextualisée. Ce processus intègre des mécanismes de recherche d'information et de génération de texte, permettant d'exploiter des sources externes pertinentes. On peut le décrire par trois étapes principales. Ces étapes sont détaillées comme suit :

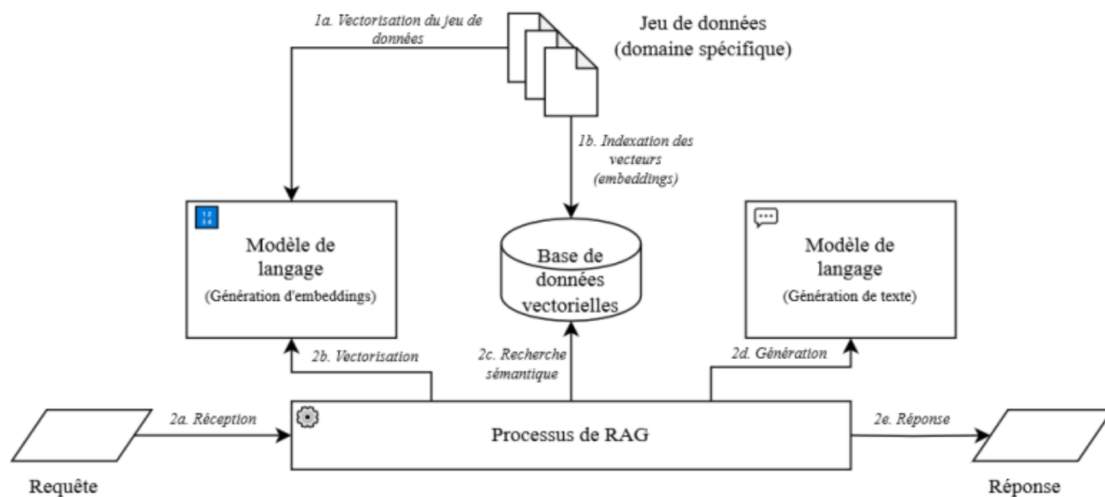


Figure 23 – Schéma de fonctionnement d'un système RAG

IV.2.1 Requête utilisateur

Le processus débute lorsqu'un utilisateur soumet une question ou une instruction en langage naturel via une interface applicative(application web, Chatbot ou API). Cette requête exprime un besoin informationnel précis, souvent lié à un domaine particulier.

IV.2.2 Recherche documentaire (Retrieval)

La requête utilisateur est transformée en une représentation est ensuite comparée aux vecteurs des documents stockés dans une base vectorielle afin d'identifier les contenus les plus proches sur le plan sémantique. Les documents les plus pertinents (**top-k**) sont alors sélectionnés pour constituer le contexte qui sera utilisé lors de la génération de la réponse.

IV.2.3 Génération de la réponse (Generation)

Les documents récupérés sont ensuite intégrés au prompt du modèle de langage (**LLM**), en complément de la requête initiale. Le modèle de langage utilise ensuite ces informations pour générer une réponse plus précises et mieux adaptée au contexte. Cette approche permet non seulement d'améliorer la pertinence des réponses , mais aussi de limiter les risques d'hallucinations en se basant sur des sources réelles.

IV.3 Composants d'un système RAG

Un système repose sur plusieurs composants clés, chacun jouant un rôle spécifique dans la chaîne de traitement.

IV.3.1 Les documents

Les documents sont la source de connaissance externe du système RAG. Ils peuvent être de nature différente :

- Document PDF,
- Pages web,
- Bases de connaissances internes,
- Fichiers texte ou bases de données structurées.

Ces documents passent par une phase de prétraitement, comprenant le nettoyage, la normalisation et le découpage en fragments (chunking), avant leur exploitation. Ce découpage permet d'assurer une granularité adaptée lors de la recherche d'information.

IV.3.2 Les embeddings

Les embeddings sont des représentations vectorielles denses qui permettent de transmettre le sens sémantique des textes. Ils sont produits à partir de modèles spécialisés (comme ceux de la famille des Sentence Transformers ou via des API d'embeddings). On transforme chaque fragment du document en un vecteur numérique de dimension fixe. De la même manière, la requête de l'utilisateur est encodée dans le même espace vectoriel, ce qui permet de mesurer la similarité sémantique entre la question et les documents.

IV.3.3 La base vectorielle

La base vectorielle constitue une pièce essentielle du système RAG. Cela permet de stocker, indexer et interroger efficacement un grand volume d'embeddings. Elle est optimisée pour des recherches par similarité (par exemple via la similarité cosinus), contrairement aux bases de données classiques. Des solutions comme Qdrant, FAISS ou Pinecone sont couramment utilisées. Elles assurent de courts temps de réponse même à partir de gros corpus, ce qui est indispensable pour une utilisation en temps réel.

IV.3.4 Le LLM (Large Language Model)

Le LLM est responsable de la génération finale de la réponse. Contrairement à un usage classique où le modèle s'appuie uniquement sur sa connaissance interne, le LLM dans un système RAG est guidé par un contexte externe fourni dynamiquement. Ce mécanisme permet :

- Améliorer la précision des réponses,
- Dédire les hallucinations,
- Actualiser facilement la base de connaissances sans réentraîner le modèle.

Le LLM agit ainsi comme un moteur de synthèse intelligent, capable de reformuler et de structurer l'information extraite des documents.

IV.4 Les types de RAG

IV.4.1 RAG naïf(simple RAG)

Le RAG naïf représente la forme la plus basique de l'approche Retrieval-Augmented Generation .Il consiste à récupérer un ensemble de documents pertinents à partir d'une base de données ,généralement via une recherche vectorielle ,puis à les transmettre au modèle de langage afin de générer une réponse .Cette approche est simple à mettre en œuvre ,mais elle produit des résultats limités lorsque les documents récupérés ne sont pas suffisamment pertinentes ou bien classés .[25]

IV.4.2 RAG avec re-ranking

Afin d'améliorer la pertinence des résultats ,certaines approches introduisent une étape de ré-ordonnancement (re-ranking) .Après la récupération initiale des documents ,un modèle supplémentaire est utilisé pour classer ces documents selon leur niveau de pertinence par rapport à la requête .Cette technique permet d'optimiser la qualité des informations fournies au modèle génératif et améliorer la précision des réponses .[26]

IV.4.3 RAG multi-requetes (Multi-query RAG)

Le RAG multi-requêtes repose sur la reformulation de la question initiale en plusieurs variantes. Ces différentes requêtes permettent d'élargir l'espace de recherche et d'augmenter les chances de récupérer des documents pertinents. Cette approche est particulièrement utile dans les contextes où les termes utilisés peuvent varier, comme dans les documents juridiques ou techniques.[27]

IV.4.4 RAG hybride (Hybride RAG)

Le RAG hybride combine deux approches complémentaires : la recherche sémantique basée sur les embeddings et la recherche lexicale traditionnelle, telle que BM25. Cette combinaison permet de tirer parti des avantages des deux méthodes, en améliorant à la fois la compréhension du sens et la correspondance exacte des mots-clés. Cette approche est particulièrement adaptée aux domaines spécialisés nécessitant une grande précision.[28]

IV.4.5 RAG conversationnel (Conversationnel RAG)

Le RAG conversationnel prend en compte l'historique des interactions entre l'utilisateur et le système. Il génère des réponses cohérentes dans un contexte de dialogue continu, en utilisant les échanges précédents pour mieux interpréter les nouvelles requêtes. Cette approche est essentielle pour les chatbots intelligents et interactifs.[29]

IV.4.6 RAG agentique (Agentic RAG)

Le RAG agentique représente une évolution avancée du RAG, où un agent intelligent est capable de planifier plusieurs étapes de recherche, de reformuler les requêtes et de valider les informations avant de générer une réponse finale. Cette approche permet d'améliorer considérablement la qualité et la fiabilité des réponses, mais elle est plus complexe à mettre en œuvre.[30]

IV.5 Comparaison :LLM seul VS LLM+RAG

Table 16 – Comparaison entre LLM et LLM+RAG

Critère	LLM	LLM+RAG
Source de connaissances	Apprises uniquement durant la phase d'entraînement	Combinaison des connaissances internes du LLM et de sources externes
Accès aux données externes	Non	Oui (bases vectorielles, documents, etc.)
Gestion des requêtes hors domaine	Faible	Très bonne (récupération contextuelle)
Hallucinations	Fréquentes (surtout avec des données rares)	Fortement réduites
Fiabilité des réponses	Limitée	Élevée
Traçabilité des informations	Absente	Possible (citations des sources récupérées)
Applications réelles	Limitées	Large éventail (finance, médical, éducation, etc.)

V Natural Language Processing (NLP)

V.1 Définition du NLP

Le traitement du langage naturel (Natural Language Processing, NLP), ou traitement automatique du langage naturel, est un sous-domaine de l'intelligence artificielle qui permet aux ordinateurs de comprendre, traiter, analyser et générer le langage humain (parlé ou écrit) de manière sémantique et utile pour des tâches spécifiques. Il combine des techniques issues de la linguistique computationnelle, de l'informatique et de l'apprentissage automatique afin d'automatiser l'interaction homme-machine à travers le langage naturel. Cette discipline englobe des tâches telles que la classification de texte, la reconnaissance d'entités nommées, la réponse aux questions et la génération de texte.[31]

V.2 Traitement NLP

V.2.1 Segmentation en phrases

La segmentation en phrases consiste à diviser un texte brut en unités syntaxiques appelées phrases. Cette étape permet de structurer le texte et de faciliter les analyses linguistiques ultérieures. Elle repose généralement sur la détection de la ponctuation, comme le point, le point d'interrogation ou le point d'exclamation.

V.2.2 Tokenisation des mots

La tokenisation est le processus de division des phrases en unités plus petites appelées tokens, généralement des mots ou des sous-mots. Cette étape transforme un texte continu en une séquence structurée exploitable par les modèles.

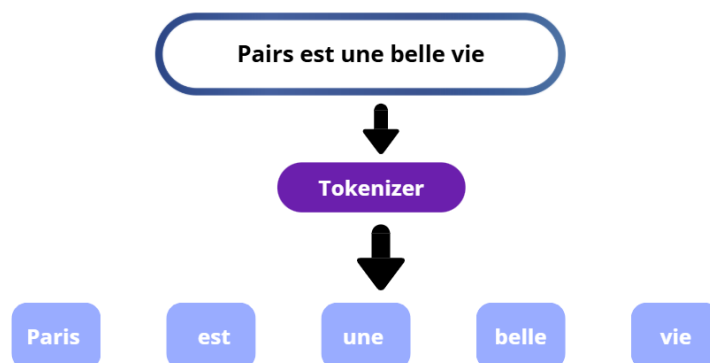


Figure 24 – Shéma de Tokenisation dans NLP

V.2.3 Stemming

Le stemming consiste à réduire les mots à leur racine lexicale en supprimant les suffixes. Par exemple, « juridique », « juridiquement » et « juridiction » peuvent être réduits à une forme simplifiée. Cette technique permet de regrouper les variantes morphologiques d'un même mot.

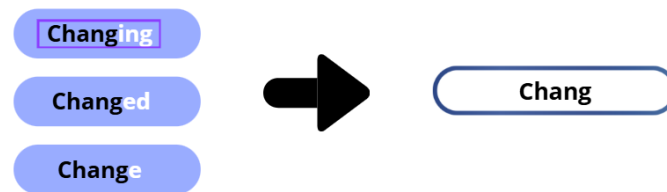


Figure 25 – Shéma de Stemming dans NLP

V.2.4 Lemmatisation

La lemmatisation est une méthode plus avancée que le stemming. Elle consiste à ramener un mot à sa forme canonique (lemme) en tenant compte de son contexte grammatical.

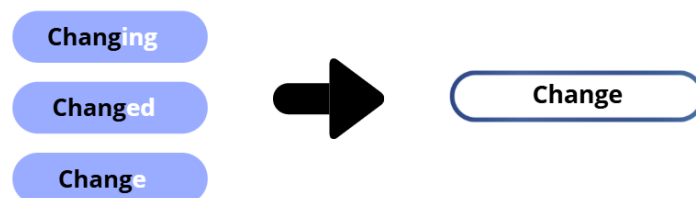


Figure 26 – Shéma de Lemmatisation dans NLP

V.2.5 Suppression des stop words

Cette étape consiste à éliminer les mots très fréquents mais peu informatifs, tels que 'le', 'la', 'et' ou 'de'. La suppression des stop words permet de réduire le bruit et d'améliorer la pertinence des analyses textuelles.

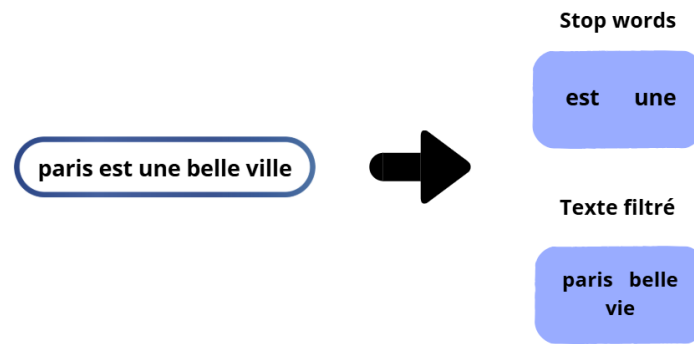


Figure 27 – Schéma de Suppression des Stop words dans NLP

V.2.6 Analyse de dépendance

L’analyse de dépendance vise à identifier les relations grammaticales entre les mots d’une phrase. Elle permet de comprendre la structure syntaxique et les relations sujet-verbe-complément, ce qui est essentiel pour l’interprétation sémantique.

V.3 Pipeline NLP classique vs pipeline basé sur LLM

Table 17 – Comparaison entre un pipeline NLP classique et un pipeline basé sur LLM

Critère	Pipeline NLP classique	Pipeline basé sur LLM
Prétraitement	Nettoyage, tokenisation, suppression des stop words, stemming ou lemmatisation	Tokenisation automatique intégrée au modèle
Feature Engineering	Manuel (Bag of Words, TF, N-grams)	Automatique via embeddings contextuels
Représentation du texte	Vecteurs statiques (TF-IDF, Word2Vec)	Embeddings contextuels dynamiques
Modèle utilisé	SVM, Naive Bayes, Random Forest	Transformers (BERT, GPT, LLaMA, etc.)
Adaptation au domaine	Requiert un entraînement spécifique	Peut être adaptée via fine-tuning ou RAG
Performance linguistique	Sensible aux variations lexicales	Meilleure compréhension sémantique
Complexité computationnelle	Faible à modérée	Élevée
Interprétabilité	Plus interprétable	Moins interprétable (boîte noire)

Ce chapitre a permis de présenter les principaux concepts liés à notre projet ,en commençant par les chatbots intelligents et leur évolution .Nous avons ensuite étudié l'approche auto-hebergée ainsi les modèles de langage de grande taille (LLM) et leur role dans les systemes conversationnels .L'approche RAG a également été abordée , en mettant évidence son fonctionnement et son apport dans l'amélioration des réponses .Par ailleurs ,les notions essentielles du traitement automatique du langage naturel (NLP) ont été introduites .Cette analyse a permis de construire une base théorique solide pour notre solution.Dans le chapitre suivant ,nous présenterons la méthodologie de travail adopté pour la réalisation de projet

Chapitre 3 : Méthodologie de travail

Ce chapitre présente la méthodologie de travail adoptée pour la réalisation du projet. Il introduit d'abord les différentes méthodologies de développement ainsi que le choix de la méthodologie retenue. Il présente également l'organisation du projet avec Scrum, les rôles des membres de l'équipe et la planification des différentes phases du projet.

I Méthodologie adoptée

Une méthodologie de gestion de projet est un ensemble de principes ,de pratiques et de processus d'organiser et de piloter le développement d'un projet informatique . il existe plusieurs types de méthodologies , classées principalement en deux grandes catégories :

- Les méthodologies traditionnelles
- les méthodologies agiles

I.1 Méthodologie traditionnel

Les méthodologies traditionnelles reposent sur une approche structurée et planifiée du développement logiciel ,ou chaque phase est clairement définie des le début du projet .Elles privilégient une documentation complété , un organisation rigoureuse est une vision globale du système avant don implémentation .Ces méthodes sont particulièrement adaptées aux projet dont les besoins sont stables et bien définis des le départ .Cependant , leur principal inconvénient réside dans leur manque de flexibilité face aux changement , ce qui peut poser problème dans environnement dynamique ou innovants.

Table 18 – Comparaison des principales méthodologies de développement

Méthodologie	Phases principales	Description
Modèle en cascade (Waterfall)	Analyse → Conception → Implémentation → Tests → Déploiement → Maintenance	Processus linéaire où chaque phase doit être terminée avant de passer à la suivante. Peu flexible face aux changements.
Modèle en V	Analyse des besoins → Conception générale → Conception détaillée → Développement ↔ Tests unitaires → Tests d'intégration → Tests système → Validation	Extension du modèle en cascade avec une correspondance entre chaque phase de développement et une phase de test.
Modèle en spirale	Planification → Analyse des risques → Développement → Évaluation (cycle répété)	Modèle itératif basé sur la gestion des risques. Le projet évolue par cycles successifs.
Modèle incrémental	Analyse globale → Développement par incréments → Tests à chaque incrément	Le produit est développé en plusieurs versions successives, chaque version ajoute des fonctionnalités.
Modèle prototype	Recueil des besoins → Création du prototype → Évaluation → Amélioration → Produit final	Création d'une version simplifiée du système afin de mieux comprendre les besoins avant le développement final.

I.2 Méthodologie agile

Les méthodologies agiles se basent sur une approche itérative et flexible du développement logiciel, centrée sur la collaboration, l'adaptation et la satisfaction du client. Contrairement aux méthodes traditionnelles, elles acceptent et encouragent les changements tout au long du projet. Le développement se fait par cycles courts, permettant de livrer rapidement des versions fonctionnelles de produit et d'intégrer régulièrement les retours des utilisateurs. Ces méthodes sont particulièrement adoptées aux projets innovants, comme les systèmes basés sur l'intelligence artificielle, où les besoins peuvent évoluer au fur et à mesure de l'avancement.

Table 19 – Comparaison des principales méthodologies agiles

Méthodologie	Phases / Pratiques principales	Description
Scrum	Planification du Sprint → Sprints (1 à 4 semaines) → Daily Scrum → Revue de Sprint → Rétrospective	Scrum est un framework agile basé sur des itérations courtes appelées sprints. Il favorise la collaboration, l'adaptation continue et l'amélioration progressive du produit à travers des réunions régulières et un backlog priorisé.
Kanban	Visualisation du flux → Limitation du travail en cours (WIP) → Gestion du flux → Amélioration continue	Kanban est une méthode agile de gestion visuelle du travail. Elle vise à optimiser le flux de production en limitant le travail en cours et en favorisant la livraison continue sans itérations fixes.
Extreme Programming (XP)	Planification → Cycles courts → Programmation en binôme → Tests continus → Intégration continue → Livraison fréquente	XP est une méthode agile orientée vers la qualité logicielle. Elle met l'accent sur les pratiques techniques telles que les tests automatisés, la programmation en binôme et les livraisons fréquentes.

I.3 Choix de la méthode à adopter

Dans le cadre de notre projet, nous avons adopté les méthodologies agiles comme approche de développement. En effet, ces méthodes sont particulièrement adaptées aux projets innovants et évolutifs, tels que les systèmes basés sur l'intelligence artificielle et les architectures RAG. Elles permettent de travailler de manière itérative, de livrer progressivement des fonctionnalités et de s'adapter facilement aux changements des besoins. De plus, les méthodologies agiles favorisent la collaboration continue avec les utilisateurs, ce qui contribue à améliorer la qualité du produit final.

Parmi les différentes méthodologies agiles, nous avons choisi d'adopter Scrum comme cadre de travail. Ce choix repose sur plusieurs points clés :

- **Adaptabilité :**

Scrum permet de s'adapter rapidement aux changements grâce à la gestion dynamique du backlog et à la réévaluation des priorités à chaque sprint.

- **Flexibilité :**

L'organisation en sprints courts offre la possibilité de modifier ou d'ajuster les fonctionnalités en cours de développement selon les besoins du projet.

- **Collaboration :**

Les réunions régulières (Daily Scrum, Sprint Review, Rétrospective) renforcent la communication et la coordination au sein de l'équipe.

- **Amélioration continue :**

Chaque sprint permet d'analyser les résultats obtenus et d'optimiser les processus pour les itérations suivantes.

- **Qualité du produit :**

Les livraisons fréquentes et les tests réguliers garantissent une meilleure fiabilité et une détection rapide des erreurs.

Ainsi, l'adoption de Scrum nous permet de structurer efficacement le développement de notre projet, tout en assurant une grande réactivité, une collaboration optimale et une amélioration continue de la solution proposée.

II Organisation de projet avec Scrum

II.1 Rôle de Scrum

Scrum est un cadre de travail agile défini officiellement dans le *Scrum Guide*. Il est utilisé pour gérer des projets complexes à travers des cycles itératifs appelés *Sprints*. Il favorise la collaboration, l'auto-organisation des équipes et l'amélioration continue, en s'appuyant sur trois piliers fondamentaux : la transparence, l'inspection et l'adaptation. Son objectif principal est de livrer rapidement et régulièrement de la valeur au client.

Composants clés de Scrum :

- **Equipe Scrum :**

L'*équipe Scrum* est composée de trois rôles principaux qui collaborent étroitement pour assurer le bon déroulement du projet :

- **Product Owner :**

représente le client et gère le Product Backlog afin de définir et prioriser les besoins du projet.

- **Scrum Master :**

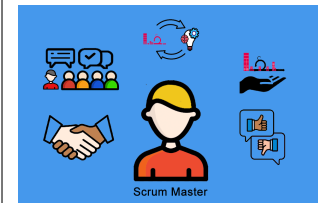
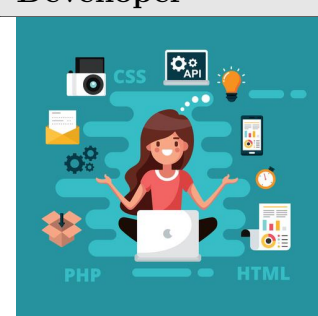
veille à l'application de la méthode Scrum ,facilite la communication et aide l'équipe à résoudre les obstacles

- **Développeurs :**

assurant la conception,le développement ,les tests et intégrations des fonctionnalités du produit .

Et voici les membres de l'équipe Scrum pour notre projet :

Table 20 – Membres de l'équipe Scrum

Product Owner	Scrum Master	Developpeur
 Figure 28 – logo de product owner	 Figure 29 – logo de scrum master	 Figure 30 – logo de développeur
Houccem saad	Zied Jaffel	Ameni Abidi et Salsabil Maali

- **Événements :**

- **Sprint** :Le *Sprint* est une période de travail limitée dans le temps, généralement entre 1 et 4 semaines. Il permet de produire un incrément fonctionnel du produit et d'assurer une livraison continue de valeur.
- **Sprint Planning** :Cette réunion, qui marque le début du *Sprint*, sert à définir les objectifs à atteindre. L'équipe sélectionne les tâches prioritaires et organise le travail en fonction de sa capacité.
- **Daily Scrum** :Le *Daily Scrum* est une réunion quotidienne courte, ne dépassant pas 15 minutes. Elle permet à l'équipe de faire le point sur l'avancement et d'identifier les éventuels obstacles.
- **Sprint Review** :À la fin de chaque *Sprint*, une revue est organisée avec les parties prenantes. L'équipe y présente le travail réalisé afin de recueillir des retours et d'ajuster le produit.
- **Sprint Retrospective** :Cette réunion interne à l'équipe permet d'évaluer le déroulement du *Sprint*. Elle vise à améliorer continuellement les pratiques et la collaboration.

- **Les artefacts :**

- **Product Backlog** :Le *Product Backlog* est une liste priorisée de toutes les fonctionnalités à développer, ainsi que des corrections et des évolutions possibles du produit.
- **Sprint Backlog** :Il contient les éléments sélectionnés du *Product Backlog* pour le *Sprint* en cours. Il sert de guide pour le travail de l'équipe durant toute l'itération.

- **Incrément** :L'*incrément* est le résultat concret du travail réalisé pendant le *Sprint*. Il doit être fonctionnel et potentiellement livrable à la fin de chaque itération.
- **Les valeurs de Scrum** :*Scrum* repose sur cinq valeurs fondamentales qui guident le comportement de l'équipe : l'engagement, le focus, l'ouverture, le respect et le courage.

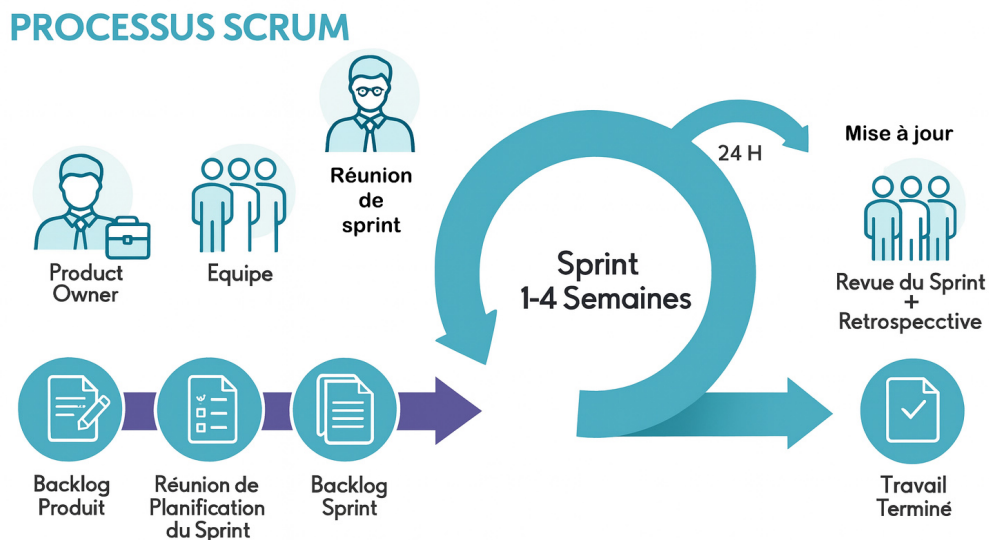


Figure 31 – Shéma de explicatif de framework SCRUM

II.2 Product backlog

a valider avec salsabil et le planification

Ce chapitre a présente la méthodologie de travail adoptée pour la realisation de notre projet.Nous avons étudié les méthodologies tradionellels et agiles ,puis justifié le choix de Scrum .Nous également presente l'organisation du projet ,les ateurs de Scrum,le Product Backlog ainsi que la planification des sprints .Cette méthodologie a permis d'assurer une bonne gestion et un suivi efficace du projet.

Chapitre 4 :Analyse et conception du système

Ce chapitre présente l'analyse et la conception du système proposé. Il décrit les différentes maquettes réalisées afin de visualiser les interfaces de l'application et les fonctionnalités offertes aux utilisateurs. Il présente également la modélisation UML à travers les différents diagrammes, ainsi que l'architecture générale de la solution et le patron de conception adopté.

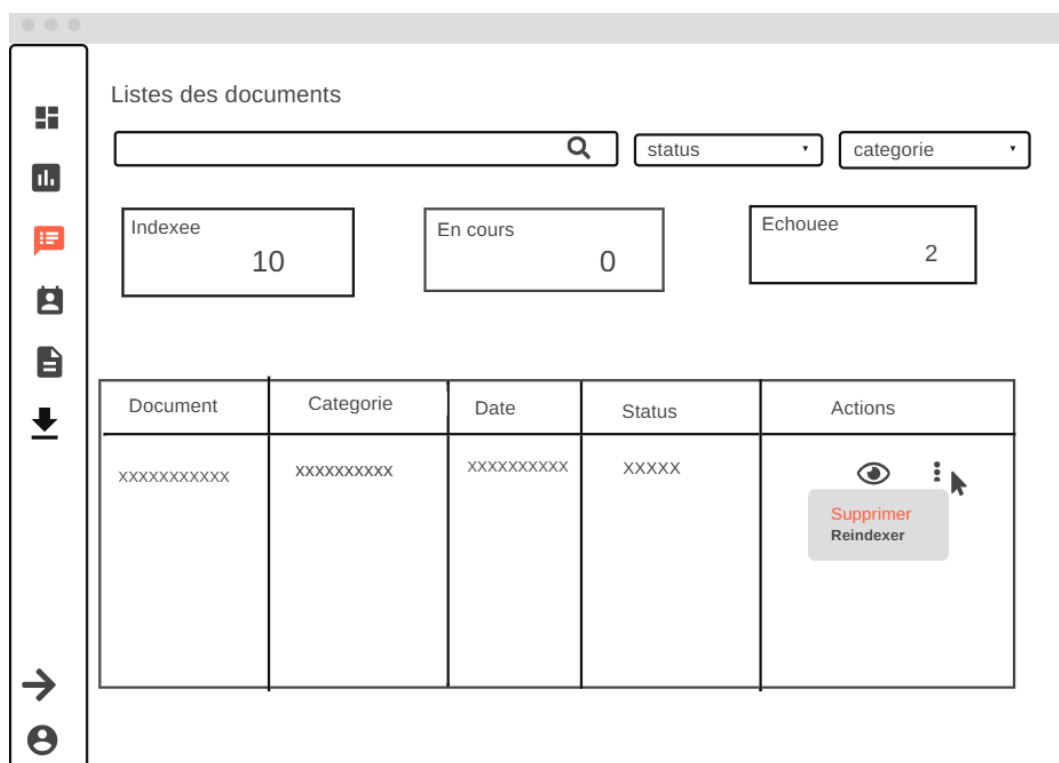
I Les Maquettes

Cette section présente les différentes maquettes réalisées pour visualiser les interfaces principales de l'application et illustrer les fonctionnalités destinées aux administrateurs et les agnets.

I.1 Les maquettes de l'administrateur

- **Listes des documents**

Cette maquette représente l'interface de gestion des documents disponible pour l'administrateur. Elle permet de consulter la liste des documents indexés, effectuer des recherches et filtrer les documents selon leur catégorie ou leur statut. L'administrateur peut également visualiser les détails d'un document, relancer son indexation ou supprimer un document depuis la plateforme.



status

categorie

Indexee

10

En cours

0

Echouee

2

Document	Categorie	Date	Status	Actions
xxxxxxxxxx	xxxxxxxxxx	xxxxxxxxxx	xxxxx	Supprimer Reindexer

Figure 32 – Maquette de "Listes des documents"

- **Importation de document**

Cette maquette illustre l'interface d'importation des documents juridiques et réglementaires dans le système. Elle permet à l'administrateur de téléverser des fichiers PDF ou Word puis de renseigner plusieurs métadonnées comme la catégorie, le type de document, le statut juridique et les dates importantes. Après la validation des informations, l'administrateur peut lancer le processus d'indexation afin d'intégrer le document dans la base documentaire intelligente.

Import document

Fichier

Drag & drop or choose File

Metadonnées

Categorie

Type de document

Status juridique

Titre

Date de publication

Date d'entre en virgueur

Version

Relation juridique

Annuler Indexer

Figure 33 – Maquette de "Importer document"

- **Suivi des réclamations**

Cette maquette présente l'interface de suivi des réclamations et des tickets des agents . Elle permet à l'administrateur de consulter les différentes réclamations selon leur état, leur priorité ou leur date de création. L'interface offre également des fonctionnalités de recherche et de suivi afin de faciliter le traitement et la gestion des demandes des agents.

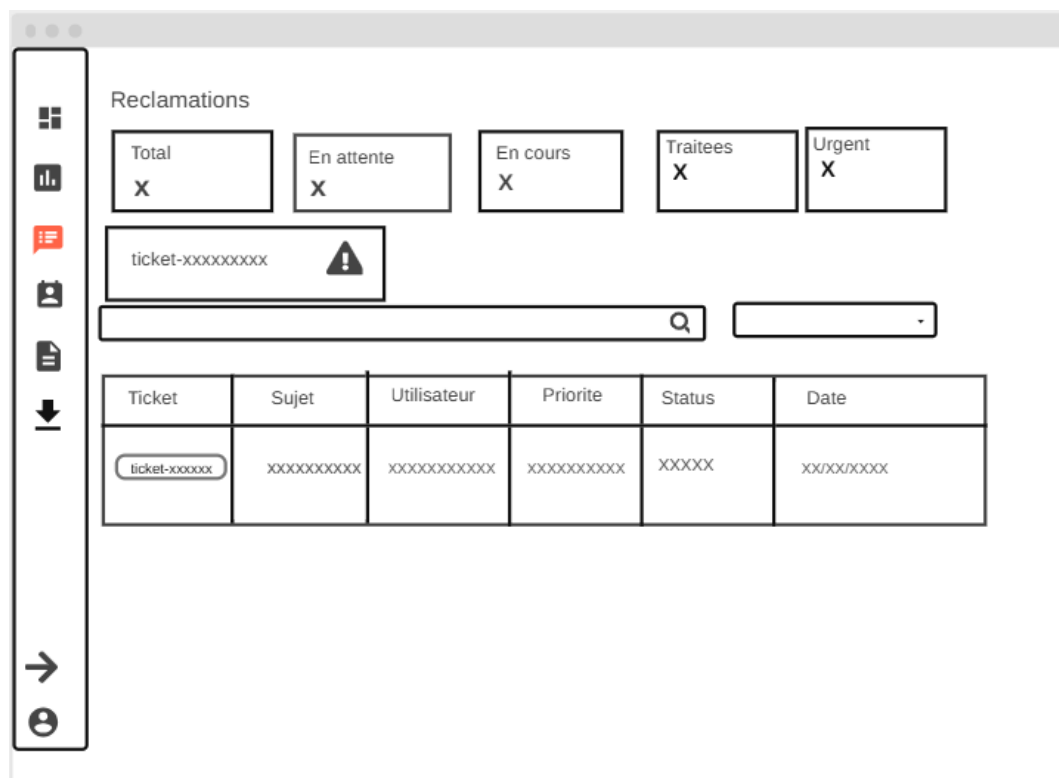


Figure 34 – Maquette de "Réclamation"

I.2 Les maquettes des agents

- **Interface de chatbot**

Cette maquette représente l'interface conversationnelle permettant aux agents de poser des questions en langage naturel concernant les documents juridiques et réglementaires. Elle permet également de consulter l'historique des conversations, effectuer des recherches dans les discussions précédentes et démarrer une nouvelle conversation avec le chatbot intelligent ou il peut archives ces conesrations.



Figure 35 – Maquette de "Chatbot utilisateur"

- **Mes réclamations**

Cette maquette présente l'interface de gestion des réclamations des utilisateurs. Elle permet aux agents de consulter leurs tickets, suivre leur statut, filtrer les réclamations selon leur état et accéder aux détails de chaque demande.

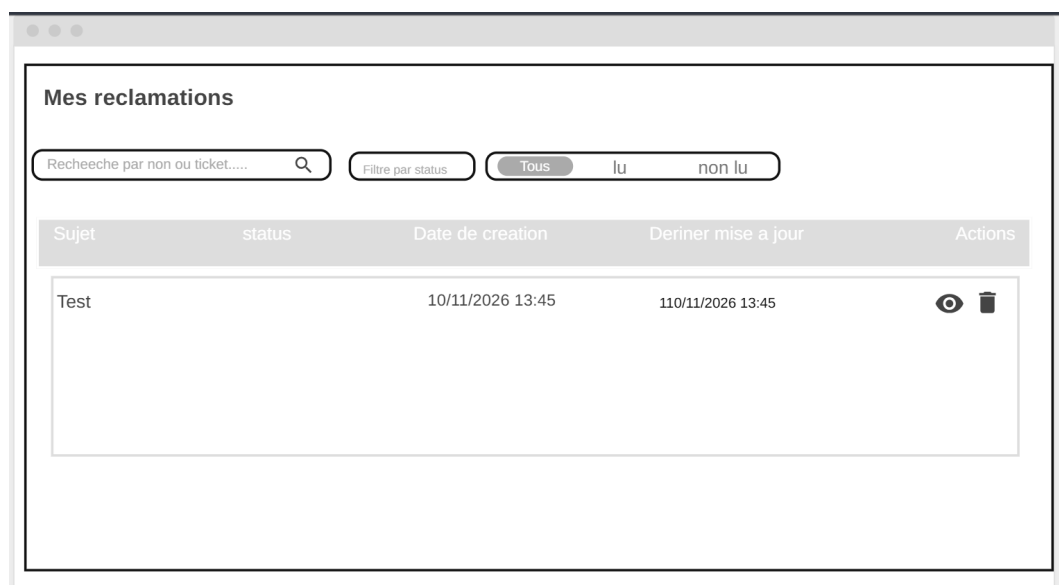


Figure 36 – Maquette de "Mes réclamations"

- **Suppression d'une réclamation**

Cette maquette illustre la fonctionnalité de suppression d'une réclamation existante. Une fenêtre de confirmation s'affiche afin de permettre à l'utilisateur de confirmer ou d'annuler l'opération avant la suppression définitive du ticket.



Figure 37 – Maquette de "Suppression de réclamation"

- **Nouvelle réclamation**

Cette maquette représente l'interface de création d'une nouvelle réclamation. Elle permet aux utilisateurs de saisir le sujet, la description, le type de problème, le niveau de priorité ainsi que d'ajouter des pièces jointes avant l'envoi de la demande.

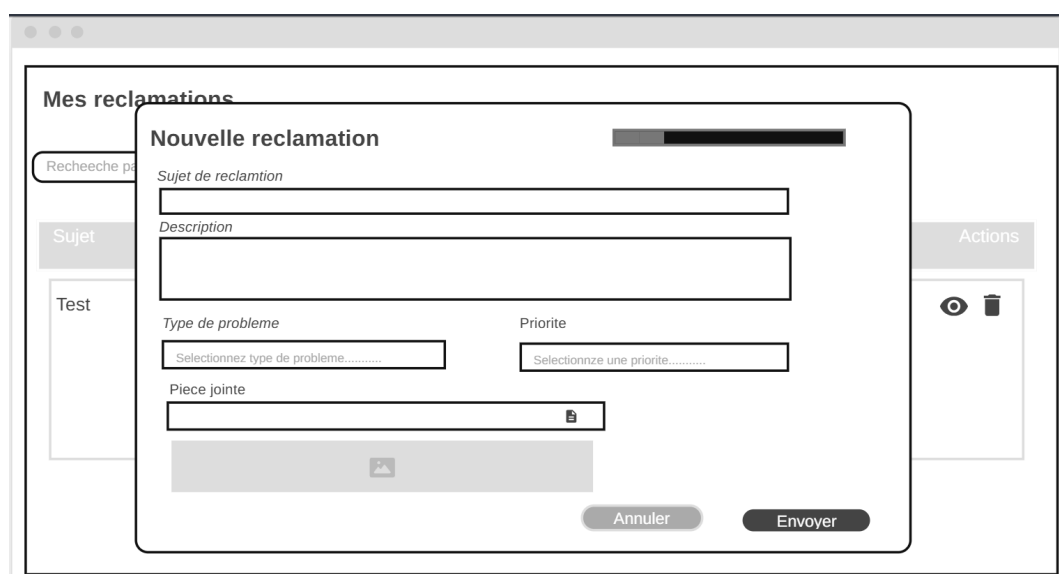


Figure 38 – Maquette de "Nouvelle réclamation"

- **Recherche documentaire**

Cette maquette présente l'interface de recherche dans les documents juridiques. Elle permet aux utilisateurs d'effectuer des recherches avancées à l'aide de filtres comme la catégorie, la période ou le nom du document afin d'obtenir des résultats plus précis.

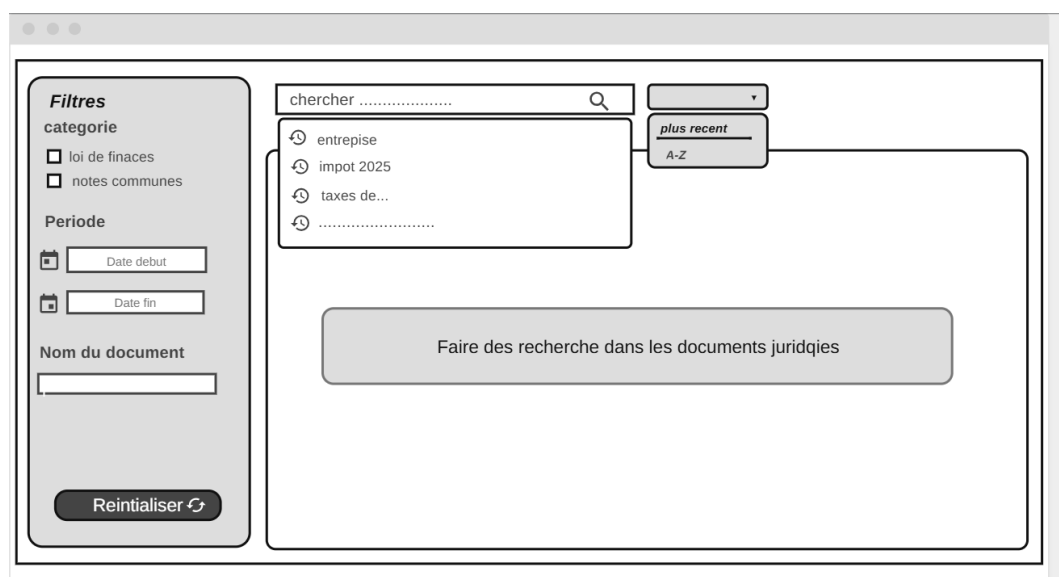


Figure 39 – Maquette de "Recherche documentaire"

- **Résultats de recherche**

Cette maquette illustre l’affichage des résultats de recherche documentaire. Les utilisateurs peuvent consulter les extraits des documents trouvés, afficher les détails complets du document et ajouter certains documents à leur liste de favoris.

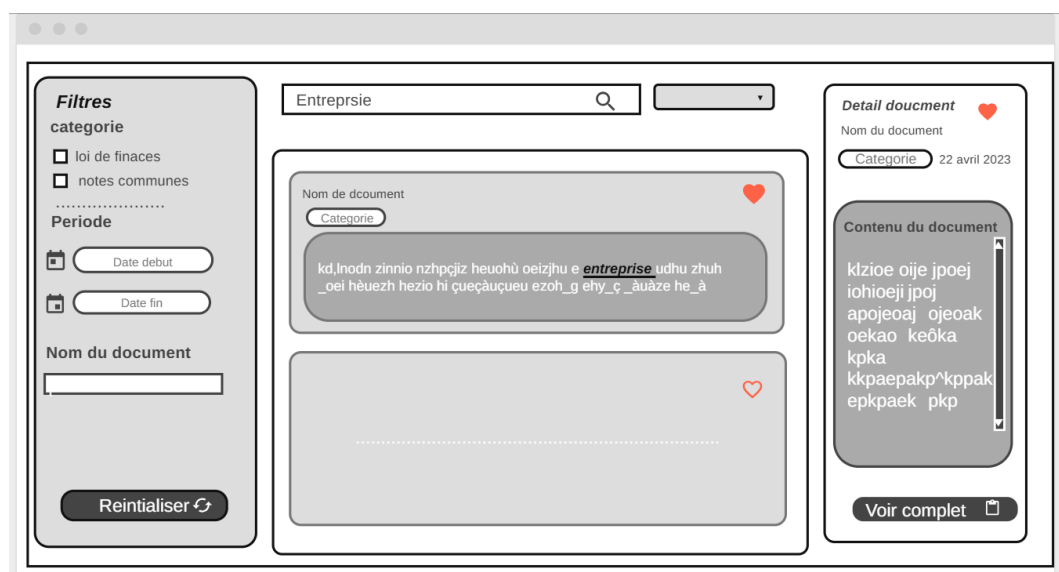


Figure 40 – Maquette de "Résultats de recherche"

- **Liste des favoris**

Cette maquette représente l’espace des documents favoris enregistrés par l’utilisateur. Elle permet de retrouver rapidement les documents importants et d’accéder directement à leur contenu détaillé.

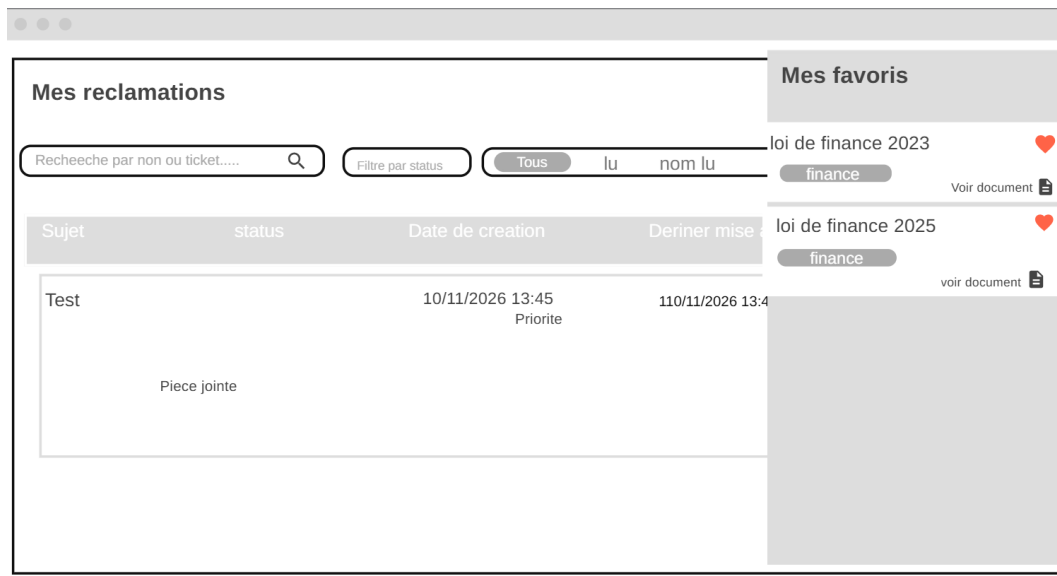


Figure 41 – Maquette de "Liste des favoris"

- **Notifications**

Cette maquette présente le système de notifications de la plateforme. Elle permet aux utilisateurs de consulter les notifications liées aux nouvelles réclamations, aux documents récemment indexés ou aux mises à jour importantes du système.

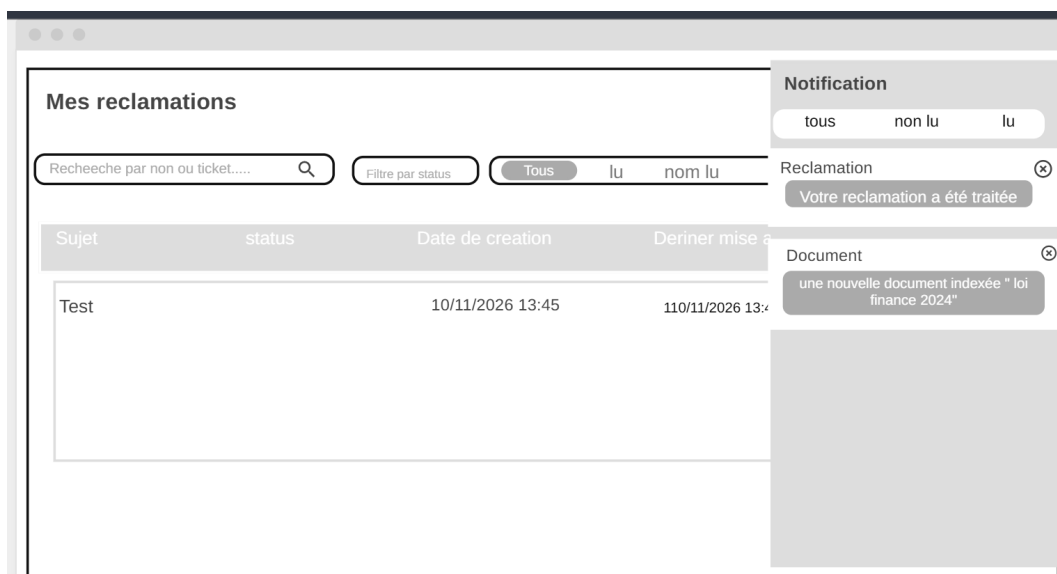


Figure 42 – Maquette de "Notifications"

II Modélisation UML

II.1 Diagramme de cas d'utilisation

II.2 Diagramme de séquences

II.2.1 Diagramme de séquence conception ""

II.2.2 Diagramme de séquence conception ""

II.2.3 Diagramme de séquence conception ""

II.2.4 Diagramme de séquence conception ""

II.2.5 Diagramme de séquence conception ""

II.3 Diagramme de Classes

II.4 Diagramme de Flux de donnée

II.5 Diagramme de Activites

III L'architecture de la solution

III.1 architecture général du projet

Pour notre projet, d'après la comparaison présentée dans l'annexe A, nous avons choisi d'adopter une architecture n-tiers, car elle permet de séparer clairement les responsabilités de l'application tout en garantissant l'évolutivité, la simplicité et la maintenabilité du système. L'architecture est structurée en 6 tiers principaux.

- **Tiers client :**

Il représente le point d'accès de l'utilisateur au système .la navigateur permet d'interagir avec l'application via une interface web sécurisée.

- **Tiers frontend :**

Développé avec *React*, *TypeScript*, *Vite* et **Tailwind CSS**, il assure l'affichage de l'interface, la navigation, l'authentification, l'importation des documents, la recherche documentaire, le chat RAG et la gestion des réclamations, tout en gérant la communication fluide avec le backend.

- **Tiers backend :**

onstruit avec FastAPI, il expose les *API REST*, applique les règles métier et gère les utilisateurs, les sessions, les documents, les notifications, les audits et les réclamations, tout en coordonnant les services internes.

- **Tiers Rag / NLP :**

responsable du traitement intelligent des documents et des questions. Il assure l'extraction du texte, le découpage des documents en chunks, la génération des embeddings, la recherche des passages pertinents et la génération des réponses.

- **Tiers de données :**

regroupe *MongoDB* et *Qdrant*. MongoDB stocke les données applicatives telles que les utilisateurs, les documents, les conversations, les notifications et les réclamations. Qdrant, utilisé comme base vectorielle, stocke les embeddings afin d'assurer une recherche sémantique rapide et pertinente.

- **Tiers services externes LLM :**

représente les modèles d'intelligence artificielle exécutés localement via *Ollama*. Ce tiers assure la génération des embeddings des documents ainsi que la production de réponses intelligentes adaptées aux questions des utilisateurs. Dans notre projet, le modèle d'embedding utilisé est *mxbai-embed-large :latest*, configuré localement afin de garantir de bonnes performances et une meilleure confidentialité des données.

III.2 Patron de conception

III.2.1 Modelé d'architecture

III.2.2 Les avantages du patron MVC

III.3 Diagramme de déploiement

Annexe 1 – Étude comparative

I. Comparaison des frameworks Frontend

Table 21 – Comparaison des différents frameworks Frontend

Critère	React	Angular	Streamlit
Type	Bibliothèque JavaScript	Framework complet	Framework Python orienté Data
Structure de projet	Flexible et modulaire	Très structuré	Simple et peu structuré
Performance UI	Excellente	Excellente	Moyenne
Adapté aux applications complexes	Oui	Oui	Non
Personnalisation UI	Très élevée	Élevée	Limitée
Adapté à la production à grande échelle	Oui	Oui	Plutôt prototype
Cas idéal	Applications interactives	Applications structurées d'entreprise	Démonstrations IA / Dashboard rapide

II. Comparaison des technologies Backend

Table 22 – Comparaison des technologies Backend

Critère	Node.js	Python	Spring Boot
Type	Environnement JavaScript côté serveur	Langage polyvalent	Framework backend robuste
Structure de projet	Flexible	Structuré	Très structuré
Performance	Excellente (asynchrone)	Très bonne	Très élevée
Adapté aux applications complexes	Oui	Oui	Oui
Personnalisation	Très élevée	Élevée	Élevée
Adapté à la production à grande échelle	Oui	Oui	Oui
Consommation des ressources	Légère	Moyenne	Plus lourde
Cas idéal	Applications API en temps réel	Applications IA / Data	Systèmes d'entreprise complexes

III. Comparaison des systèmes de gestion de bases de données

Table 23 – Comparaison entre MongoDB et PostgreSQL

Critère	MongoDB	PostgreSQL
Type	NoSQL orienté documents	Base relationnelle SQL
Structure des données	Flexible (schéma dynamique)	Structurée (schéma fixe)
Performance en lecture rapide	Très bonne	Très bonne
Transactions complexes	Limitée	Excellente gestion
Scalabilité	Bonne	Bonne
Adapté aux projets RAG	Oui (métadonnées, logs)	Oui (si données relationnelles)
Cas idéal	Applications évolutives	Systèmes transactionnels

IV. Comparaison des bases de données vectorielles et outils de similarité

Table 24 – Comparaison entre Qdrant, FAISS et ChromaDB

Critère	Qdrant	FAISS	ChromaDB
Type	Base vectorielle complète	Bibliothèque de similarité	Base vectorielle légère
Auto-hébergement	Oui	Oui	Oui
Performance de recherche	Très élevée	Très élevée	Élevée
Scalabilité	Bonne	Bonne	Moyenne
API intégrée	Oui (REST)	Non natif	Oui
Cas idéal	Système RAG en production	Recherche haute performance (cloud et local)	Projets RAG simples

V. Comparaison des modèles de langage (LLM)

Table 25 – Comparaison entre Ollama, Claude et GPT

Critère	Ollama	Claude	GPT
Type	Runtime local LLM	Modèle propriétaire	Modèle propriétaire
Auto-hébergement	Oui	Non	Non
Performance	Bonne	Très élevée	Très élevée
Coût	Gratuit (local)	Payant	Payant
Personnalisation	Élevée	Limitée	Limitée
Cas idéal	Projets locaux sécurisés	Applications éthiques contrôlées	Applications cloud avancées

III.4 Comparaison des différents types d'architecture logicielle

Table 26 – Comparaison entre l'architecture Microservices et l'architecture n-tiers

Microservices	Architecture n-tiers
Ensemble de services indépendants communiquant via des API	Sépare l'application en plusieurs couches, chacune ayant un rôle précis
Souple et scalable	Facilite la maintenance, l'évolution et la compréhension du système
Forte indépendance entre les composants	Bonne organisation du code
Complexe à concevoir, orchestrer et déployer	Permet la scalabilité et la répartition du travail
Nécessite une gestion avancée des services, du monitoring et de la sécurité interservices	Communication entre les tiers via des API ou des protocoles réseau